第6章　快速查询的秘籍——B+树索引

前文详细唠叨了 InnoDB 数据页的 7 个组成部分，我们知道了各个数据页可以组成一个双向链表，而每个数据页中的记录会按照主键值从小到大的顺序组成一个单向链表。每个数据页都会为存储在它里面的记录生成一个页目录，在通过主键查找某条记录的时候可以在页目录中使用二分法快速定位到对应的槽，然后再遍历该槽对应分组中的记录即可快速找到指定的记录（如果你对这段话有一丁点儿疑惑，那么接下来的部分不适合你，返回去看一下数据页结构吧）。页和记录的关系示意图如图 6-1 所示。

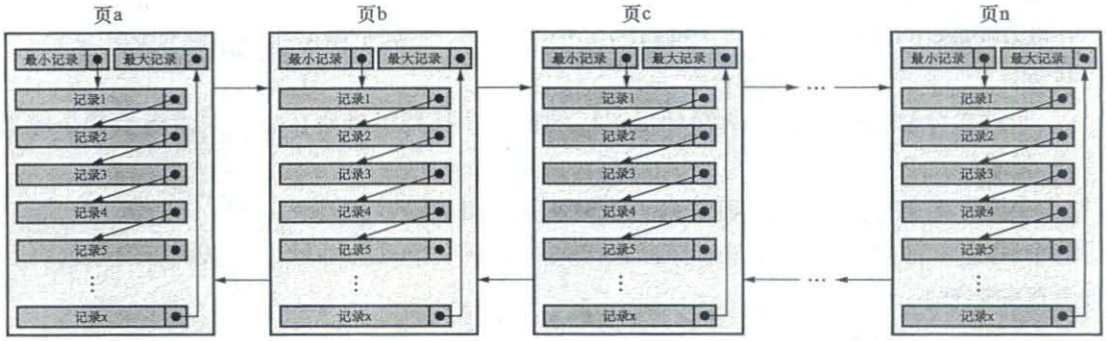

图 6-1　页和记录的关系示意图

其中，页 a、页 b、页 c……页 n 这些页可以不在物理结构上相连，只要通过双向链表相关联即可。

6.1　没有索引时进行查找

本章的主题是索引。在正式介绍索引之前，我们需要了解一下在没有索引时是怎么查找记录的。为了方便大家理解，我们先只唠叨搜索条件为某个列等于某个常数的情况，比如下面这样：

```
SELECT [查询列表] FROM 表名 WHERE 列名 = xxx;
```

6.1.1　在一个页中查找

假设目前表中的记录比较少，所有的记录都可以存放到一个页中。在查找记录时，可以根据搜索条件的不同分为两种情况。

- 以主键为搜索条件：这个查找过程我们已经很熟悉了，可以在页目录中使用二分法快速定位到对应的槽；然后再遍历该槽对应分组中的记录，即可快速找到指定的记录。
- 以其他列作为搜索条件：对非主键列的查找可就不这么幸运了，因为在数据页中并没

有为非主键列建立所谓的页目录，所以无法通过二分法快速定位相应的槽。在这种情况下，只能从 Infimum 记录开始依次遍历单向链表中的每条记录，然后对比每条记录是否符合搜索条件。很显然，这种查找的效率非常低。

6.1.2 在很多页中查找

在很多时候，表中存放的记录都是非常多的，需要用到好多的数据页来存储这些记录。在很多页中查找记录可以分为两个步骤：

- 定位到记录所在的页；
- 从所在的页内查找相应的记录。

在没有索引的情况下，无论是根据主键列还是其他列的值进行查找，由于我们不能快速地定位到记录所在的页，所以只能从第一页沿着双向链表一直往下找。在每一页中我们根据刚刚唠叨过的查找方式去查找指定的记录。因为要遍历所有的数据页，所以这种方式显然是超级耗时的。如果一个表有 1 亿条记录，使用这种方式去查找记录，估计要等到猴年马月才能查找到结果。所以人们都在期盼一种能高效完成搜索的方法，索引“同志”就要亮相登台了。

6.2 索引

为了故事的顺利发展，我们先建一个表：

```
mysql> CREATE TABLE index_demo(
    ->     c1 INT,
    ->     c2 INT,
    ->     c3 CHAR(1),
    ->     PRIMARY KEY(c1)
    -> ) ROW_FORMAT = COMPACT;
Query OK, 0 rows affected (0.03 sec)
```

这个新建的 index_demo 表中有 2 个 INT 类型的列、1 个 CHAR(1) 类型的列，而且我们规定 c1 列为主键。这个表使用 COMPACT 行格式来实际存储记录。为了理解上的方便，我们简化了 index_demo 表的行格式示意图，如图 6-2 所示。

图 6-2 index_demo 表的行格式示意图

下面只讲解图 6-2 中展示的这几个部分。

- record_type：记录头信息的一项属性，表示记录的类型。其中，0 表示普通记录；2 表示 Infimum 记录；3 表示 Supremum 记录；1 还没用过，等会再说。

- next_record：记录头信息的一项属性，表示从当前记录的真实数据到下一条记录的真实数据的距离。为了方便大家理解，我们会用箭头来表明下一条记录是谁。
- 各个列的值：这里只展示在 index_demo 表中的 3 个列，分别是 c1、c2 和 c3。
- 其他信息：除了上述 3 种信息以外的所有信息，包括其他隐藏列的值以及记录的额外信息。

为了节省篇幅，后文的示意图会把记录中的“其他信息”部分省略掉，因为它占地方，并且也没有什么观赏效果。另外，我觉得把记录竖着看感觉更好，所以，记录格式示意图的“其他信息”去掉后并竖起来的效果如图 6-3 所示。

把一些记录放到页里边的示意图如图 6-4 所示。

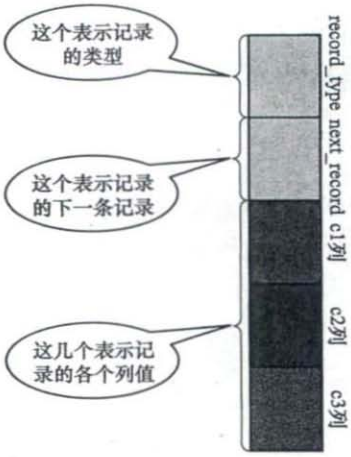

图 6-3　竖放记录的效果

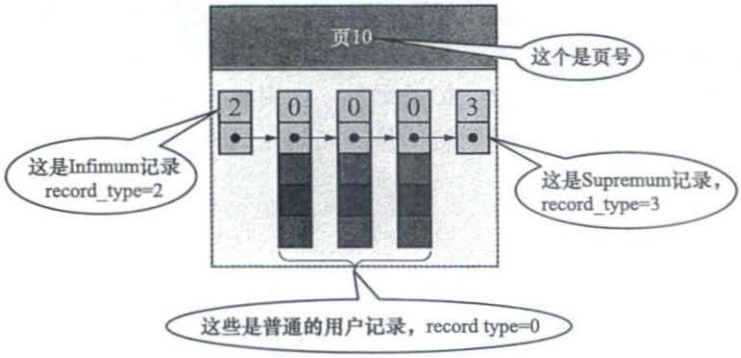

图 6-4　记录放到页里边的示意图

6.2.1　一个简单的索引方案

回到正题，我们在根据某个搜索条件查找一些记录时，为什么要遍历所有的数据页呢？原因是各个页中的记录并没有规律，我们并不知道搜索条件会匹配哪些页中的记录，所以不得不依次遍历所有的数据页。如果想快速定位到需要查找的记录在哪些数据页中，该咋办？还记得我们为了根据主键值快速定位一条记录在页中的位置而设立的页目录么？我们也可以想办法为快速定位记录所在的数据页而建立一个别的目录，在建这个目录的过程中必须完成两件事儿。

1．下一个数据页中用户记录的主键值必须大于上一个页中用户记录的主键值。

为了故事的顺利发展，我们这里需要做一个假设：每个数据页最多能存放 3 条记录（实际上一个数据页非常大，可以存放好多记录）。有了这个假设之后，我们向 index_demo 表插入 3 条记录：

```
mysql> INSERT INTO index_demo VALUES(1, 4, 'u'), (3, 9, 'd'), (5, 3, 'y');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

那么，这些记录已经按照主键值的大小串联成一个单向链表了，如图 6-5 所示。

从图 6-5 可以看出，index_demo 表中的 3 条记录都被插入到编号为 10 的数据页中。此时再插入一条记录：

```
mysql> INSERT INTO index_demo VALUES(4, 4, 'a');
Query OK, 1 row affected (0.00 sec)
```

因为页 10 最多只能放 3 条记录，所以我们不得不再分配一个新页，如图 6-6 所示。

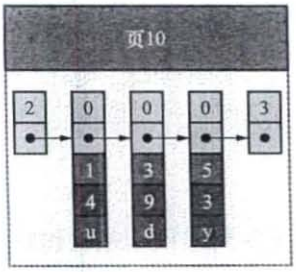

图 6-5 记录组成的单向链表

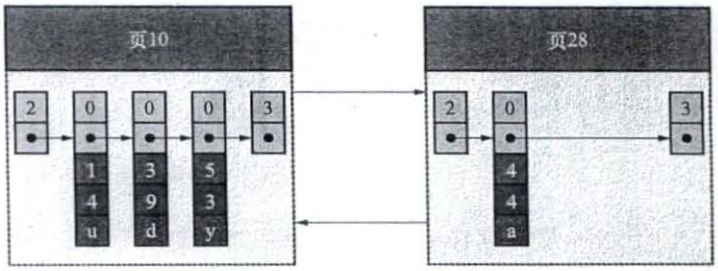

图 6-6 为记录分配新页

咦？怎么分配的页号是 28 呀，不应该是 11 么？再强调一遍，新分配的数据页编号可能并不是连续的，也就是说我们使用的这些页在磁盘上可能并不挨着（不过设计 InnoDB 的大叔会尽量让这些页面相邻，这个问题我们会在表空间的章节中详细唠叨）。它们只是通过维护上一页和下一页的编号而建立了链表关系。另外，页 10 中用户记录最大的主键值是 5，而页 28 中有一条记录的主键值是 4，因为 5 > 4，所以这就不符合“下一个数据页中用户记录的主键值必须大于上一个页中用户记录的主键值”的要求，所以在插入主键值为 4 的记录时需要伴随着一次记录移动，也就是把主键值为 5 的记录移动到页 28 中，再把主键值为 4 的记录插入到页 10 中。这个过程的示意图如图 6-7 所示。

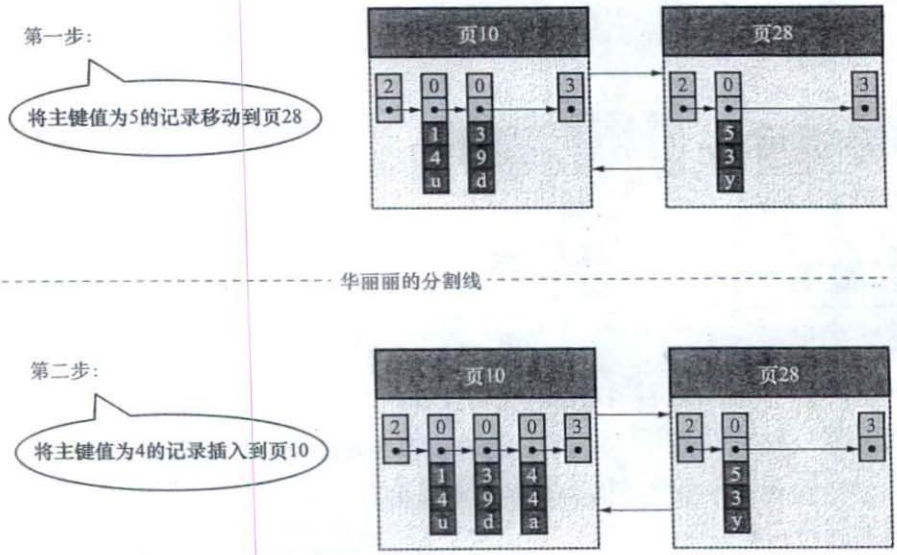

图 6-7 为记录分配新页的过程

这个过程表明，在对页中的记录进行增删改操作的过程中，我们必须通过一些诸如记录移动的操作来始终保证这个状态一直成立：下一个数据页中用户记录的主键值必须大于上一个页中用户记录的主键值。这个过程也可以称为页分裂。

2．给所有的页建立一个目录项。

由于数据页的编号可能并不是连续的，所以在向 index_demo 表中插入许多条记录后，可能会形成如图 6-8 所示的效果。

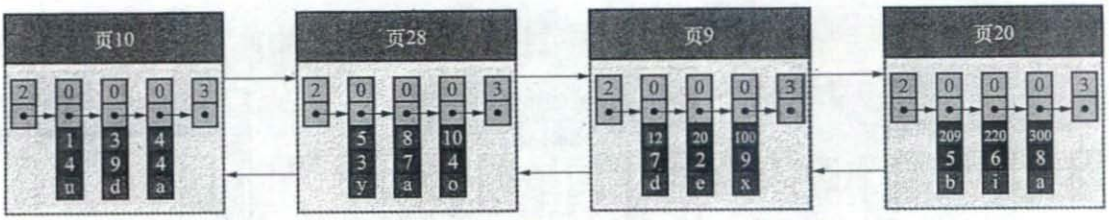

图 6-8 向 index_demo 表中插入许多条记录后的效果

由于这些大小为 16KB 的页在磁盘上可能并不挨着，如果想从这么多页中根据主键值快速定位某些记录所在的页，就需要给它们编制一个目录，每个页对应一个目录项，每个目录项包括下面两个部分：

- 页的用户记录中最小的主键值，用 key 来表示；
- 页号，用 page_no 表示。

所以我们为上面几个页编制的目录如图 6-9 所示。

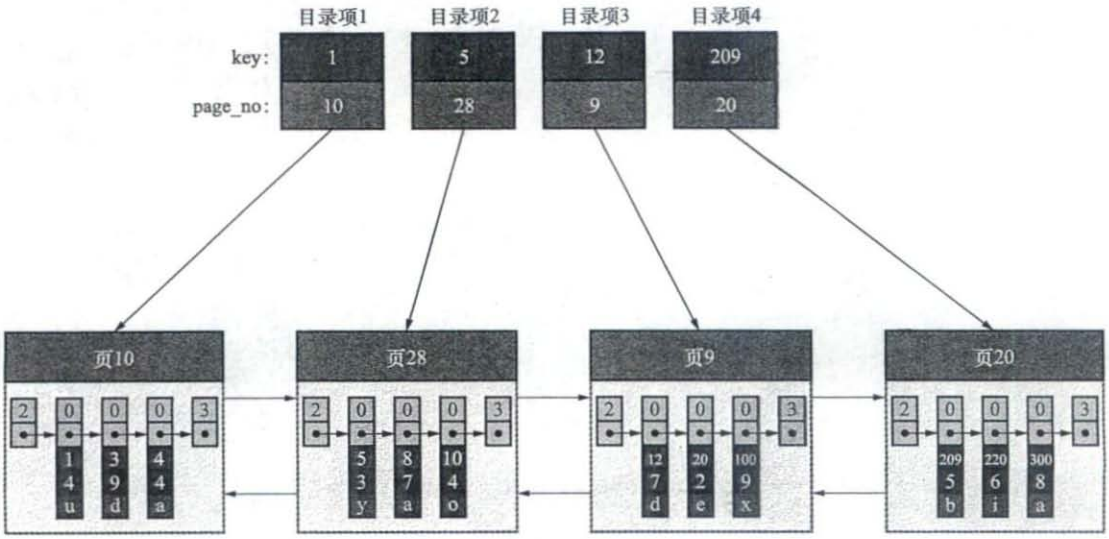

图 6-9 为页编制目录

以页 28 为例，它对应目录项 2，这个目录项中包含着该页的页号 28 以及该页中用户记录的最小主键值 5。我们只需要把几个目录项在物理存储器上连续存储，比如把它们放到一个数组中，就可以实现根据主键值快速查找某条记录的功能了。比如，我们想查找主键值为 20 的记录，具体查找过程分两步。

1. 先从目录项中根据二分法快速确定出主键值为 20 的记录在目录项 3 中（因为 12 < 20 < 209），它对应的页是页 9。
2. 再根据前文讲的在页中查找记录的方式去页 9 中定位具体的记录。

至此，针对数据页编制的简易目录就搞定了。刚才忘记说了，这个目录有一个别名，称为索引。

6.2.2 InnoDB 中的索引方案

之所以说刚才为每个数据页制作目录项的过程是一个简易的索引方案，是因为我们在根据

主键值进行查找时，为了使用二分法快速定位具体的目录项，而假设所有目录项都可以在物理存储器上连续存储。但是这样做有下面几个问题。

- InnoDB 使用页作为管理存储空间的基本单位，也就是最多只能保证 16KB 的连续存储空间。虽然一个目录项占用不了多大的存储空间，但是架不住表中记录越来越多。此时需要非常大的连续的存储空间才能把所有的目录项都放下，这对记录数量非常多的表来说是不现实的。
- 我们时常会对记录执行增删改操作，假设我们把页 28 中的记录都删除，页 28 也就没有了存在的必要。这也就意味着目录项 2 也没有了存在的必要，这就需要把目录项 2 后的目录项都向前移动一下。这种牵一发而动全身的设计不是什么好主意。又或者不移动目录项 2，而是将其作为冗余放在目录项列表中，从而浪费了很多存储空间。

所以，设计 InnoDB 的大叔需要一种可以灵活管理所有目录项的方式。他们发现这些目录项其实与用户记录长得很像，只不过目录项中的两个列是主键和页号而已，所以他们灵光乍现，复用了之前存储用户记录的数据页来存储目录项。为了与用户记录进行区分，我们把这些用来表示目录项的记录称为目录项记录。那么，InnoDB 是怎么区分一条记录是普通的用户记录还是目录项记录呢？大家别忘了记录头信息中的 record_type 属性，它的各个取值代表的意思如下。

- 0：普通的用户记录。
- 1：目录项记录。
- 2：Infimum 记录。
- 3：Supremum 记录。

原来这个值为 1 的 record_type 是这个意思。我们把前面使用到的目录项放到数据页中，如图 6-10 所示。

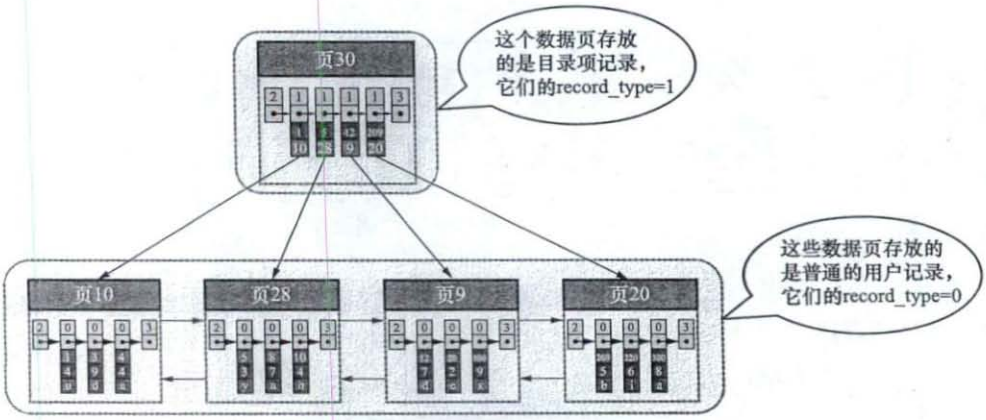

图 6-10 将目录项放到数据页中的效果

从图 6-10 可以看出，我们新分配了一个编号为 30 的页来专门存储目录项记录。这里再次强调一下目录项记录和普通的用户记录的不同点。

- 目录项记录的 record_type 值是 1，普通用户记录的 record_type 值是 0。
- 目录项记录只有主键值和页的编号两个列，而普通用户记录的列是用户自己定义的，可能包含很多列，另外还有 InnoDB 自己添加的隐藏列。
- 我们在前面唠叨记录头信息时说过一个名为 min_rec_flag 的属性，只有目录项记录的

min_rec_flag 属性才可能为 1，普通用户记录的 min_rec_flag 属性都是 0。

除了上述几点外，这两者就没啥差别了：它们用的是一样的数据页（页面类型都是 0x45BF，这个属性在 File Header 中）；页的组成结构也是一样的（就是我们前面介绍过的 7 个部分）；都会为主键值生成 Page Directory（页目录），从而在按照主键值进行查找时可以使用二分法来加快查询速度。

现在以查找主键为20的记录为例，根据某个主键值去查找记录的步骤可以大致拆分为两步。

1. 先到存储目录项记录的页（也就是页 30）中通过二分法快速定位到对应的目录项记录，因为 12 < 20 < 209，所以定位到对应的用户记录所在的页就是页 9。
2. 再到存储用户记录的页 9 中根据二分法快速定位到主键值为 20 的用户记录。

虽然说目录项记录中只存储主键值和对应的页号，比用户记录需要的存储空间小多了，但是毕竟一个页只有 16KB 大小，能存放的目录项记录也是有限的。如果表中的数据太多，以至于一个数据页不足以存放所有的目录项记录，该咋办呢？

当然是再多整一个存储目录项记录的页了。为了让大家更好地理解新分配一个存储目录项记录的页的过程，我们假设一个存储目录项记录的页最多只能存放 4 条目录项记录（请注意这是假设，真实情况下可以存放好多条）。如果此时再向图 6-10 中插入一条主键值为 320 的用户记录，那就需要分配一个新的存储目录项记录的页了，如图 6-11 所示。

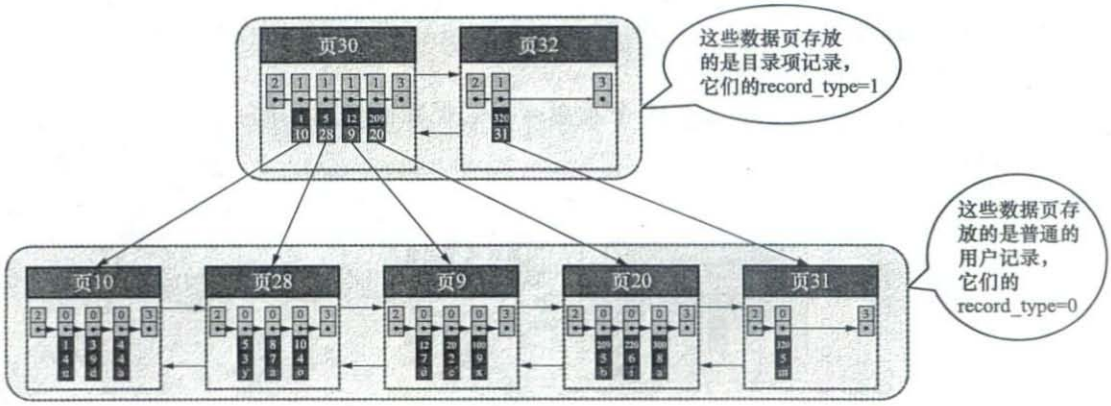

图 6-11　分配新的数据页

从图 6-11 可以看出，在插入了一条主键值为 320 的用户记录之后，需要两个新的数据页：

- 为存储该用户记录而新生成了页 31；
- 因为存储目录项记录的页 30 的容量已满（前面假设每个页只能存储 4 条目录项记录），所以不得不需要一个新的页 32 来存放页 31 对应的目录项目录。

现在因为存储目录项记录的页不止一个，此时如果想根据主键值查找一条用户记录，则大致需要 3 个步骤。以查找主键值为 20 的记录为例，具体如下。

步骤 1. 确定存储目录项记录的页。

现在存储目录项记录的页有两个，即页 30 和页 32。又因为页 30 表示的目录项记录主键值的范围是 [1, 320)，页 32 表示的目录项记录主键值不小于 320，所以主键值为 20 的记录对应的目录项记录在页 30 中。

步骤 2. 通过存储目录项记录的页确定用户记录真正所在的页。

前文已经讲过如何在一个存储目录项记录的页中通过主键值定位一条目录项记录，因此这里不再赘述。

步骤 3. 在真正存储用户记录的页中定位到具体的记录。

在一个存储用户记录的页中通过主键值定位一条用户记录的方式已经说过好多遍了。你要是还不会，我就……我就求你翻到上一章多看几遍有关数据页结构的内容了。

那么问题来了，在这个查询步骤的步骤 1 中，我们需要定位存储目录项记录的页，但是这些页在存储空间中也可能不挨着。如果表中的数据非常多，则会产生很多存储目录项记录的页，那我们怎么根据主键值快速定位一个存储目录项记录的页呢？其实也简单，为这些存储目录项记录的页再生成一个更高级的目录，就像是一个多级目录一样，大目录里嵌套小目录，小目录里才是实际的数据。所以，现在各个页的示意图如图 6-12 所示。

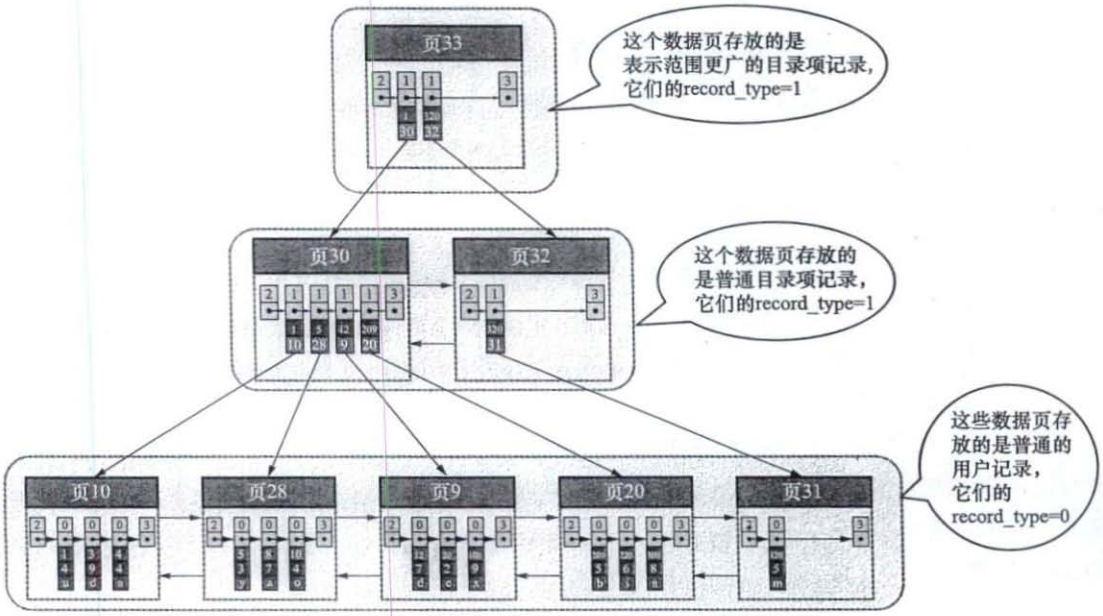

图 6-12 生成存储更高级目录项记录的数据页

在图 6-12 中，我们生成了一个存储更高级目录项记录的页 33。这个页中的两条记录分别代表页 30 和页 32。如果用户记录的主键值在 [1, 320) 之间，则到页 30 中查找更详细的目录项记录；如果主键值不小于 320，就到页 32 中查找更详细的目录项记录。随着表中记录的增加，这个目录的层级会继续增加，如果简化一下，那么可以用图 6-13 来描述它。

大家看，这玩意儿像不像一棵倒过来的树呢——上面是树根，下面是树叶！其实这是一种组织数据的形式，或者说是一种数据结构，它的名称是 B+ 树。

无论是存放用户记录的数据页，还是存放目录项记录的数据页，我们都把它们存放到 B+ 树这个数据结构中。我们也将这些数据页称为 B+ 树的节点。从图 6-12 可以看出，我们真正的用户记录其实都存放在 B+ 树最底层的节点上，这些节点也称为叶子节点或叶节点。其余用来存放目录项记录的节点称为非叶子节点或者内节点，其中 B+ 树最上边的那个节点也称为根节点。

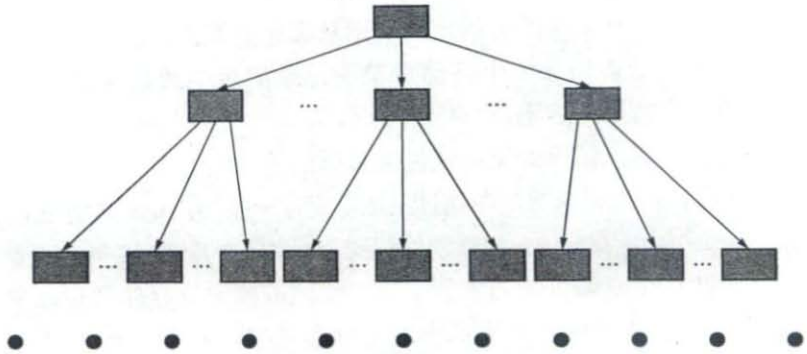

图 6-13 B+ 树

从图 6-13 可以看出，一个 B+ 树的节点其实可以分成好多层。设计 InnoDB 的大叔为了讨论方便，规定最下面的那层（也就是存放用户记录的那层）为第 0 层，之后层级依次往上加。在之前的讨论中，我们做了一个非常极端的假设：存放用户记录的页最多存放 3 条记录，存放目录项记录的页最多存放 4 条记录。其实在真实环境中，一个页存放的记录数量是非常大的。假设所有存放用户记录的叶子节点所代表的数据页可以存放 100 条用户记录，所有存放目录项记录的内节点所代表的数据页可以存放 1,000 条目录项记录，那么：

- 如果 B+ 树只有 1 层，也就是只有 1 个用于存放用户记录的节点，则最多能存放 100 条用户记录；
- 如果 B+ 树有 2 层，最多能存放 1,000 × 100 = 100,000 条用户记录；
- 如果 B+ 树有 3 层，最多能存放 1,000 × 1,000 × 100 = 100,000,000 条用户记录；
- 如果 B+ 树有 4 层，最多能存放 1,000 × 1,000 × 1,000 × 100 = 100,000,000,000 条用户记录。（这么多的记录！）

你的表里能存放 100,000,000,000 条记录么？所以在一般情况下，我们用到的 B+ 树都不会超过 4 层。这样一来，在通过主键值去查找某条记录时，最多只需要进行 4 个页面内的查找（查找 3 个存储目录项记录的页和 1 个存储用户记录的页）。又因为在每个页面内存在 Page Directory（页目录），所以在页面内也可以通过二分法快速定位记录。

小贴士

我们在唠叨数据页的 Page Header 部分时介绍过一个名为 PAGE_LEVEL 的属性，它就代表着这个数据页作为节点在 B+ 树中的层级。

1. 聚簇索引

前面介绍的 B+ 树本身就是一个目录，或者说本身就是一个索引，它有下面两个特点。

- 使用记录主键值的大小进行记录和页的排序，这包括 3 方面的含义。
  - 页（包括叶子节点和内节点）内的记录按照主键的大小顺序排成一个单向链表，页内的记录被划分成若干个组，每个组中主键值最大的记录在页内的偏移量会被当作槽依次存放在页目录中（当然 Supremum 记录比任何用户记录都大），我们可以在页目录中通过二分法快速定位到主键列等于某个值的记录。
  - 各个存放用户记录的页也是根据页中用户记录的主键大小顺序排成一个双向链表。

- 存放目录项记录的页分为不同的层级，在同一层级中的页也是根据页中目录项记录的主键大小顺序排成一个双向链表。

- B+ 树的叶子节点存储的是完整的用户记录。所谓完整的用户记录，就是指这个记录中存储了所有列的值（包括隐藏列）。

我们把具有这两个特点的 B+ 树称为聚簇索引，所有完整的用户记录都存放在这个聚簇索引的叶子节点处。这种聚簇索引并不需要我们在 MySQL 语句中显式地使用 INDEX 语句去创建（后边会介绍索引相关的语句），InnoDB 存储引擎会自动为我们创建聚簇索引。另外有趣的一点是，在 InnoDB 存储引擎中，聚簇索引就是数据的存储方式（所有的用户记录都存储在了叶子节点），也就是所谓的“索引即数据，数据即索引”。

2. 二级索引

大家是否发现，聚簇索引只能在搜索条件是主键值时才能发挥作用，原因是 B+ 树中的数据都是按照主键进行排序的。如果我们想以别的列作为搜索条件该咋办呢？难道只能从头到尾沿着链表依次遍历记录么？

不！我们可以多建几棵 B+ 树，并且不同 B+ 树中的数据采用不同的排序规则。比如，我们用 c2 列的大小作为数据页、页中记录的排序规则，然后再建一棵 B+ 树，如图 6-14 所示。

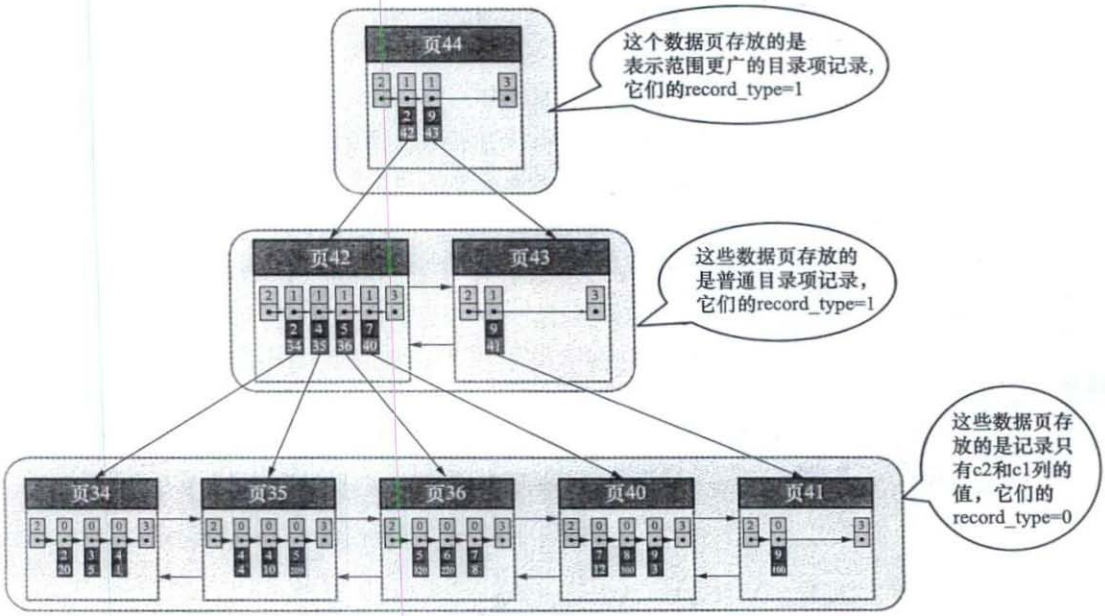

图 6-14 新建 B+ 树

这个 B+ 树与前文介绍的聚簇索引有几处不同。

- 使用记录 c2 列的大小进行记录和页的排序，这包括 3 方面的含义。
  - 页（包括叶子节点和内节点）内的记录是按照 c2 列的大小顺序排成一个单向链表，页内的记录被划分成若干个组，每个组中 c2 列值最大的记录在页内的偏移量会被当作槽依次存放在页目录中（当然规定 Supremum 记录比任何用户记录都大），我们可以在页目录中通过二分法快速定位到 c2 列等于某个值的记录。

  - 各个存放用户记录的页也是根据页中记录的 c2 列大小顺序排成一个双向链表。
  - 存放目录项记录的页分为不同的层级，在同一层级中的页也是根据页中目录项记录的 c2 列大小顺序排成一个双向链表。
- B+ 树的叶子节点存储的并不是完整的用户记录，而只是 c2 列 + 主键这两个列的值。
- 目录项记录中不再是主键 + 页号的搭配，而变成了 c2 列 + 页号的搭配。

现在，比方说我们想查找满足搜索条件 c2=4 的记录，就可以使用刚刚建好的这棵 B+ 树了。不过我们这里需要注意一下，因为 c2 列并没有唯一性约束，也就是说满足搜索条件 c2=4 的记录可能有很多条，其实我们只需要在该 B+ 树的叶子节点处定位到第一条满足搜索条件 c2=4 的那条记录，然后沿着由记录组成的单向链表一直向后扫描即可。另外，各个叶子节点组成了双向链表，搜索完了本页面的记录后可以很顺利地跳到下一个页面中的第一条记录，然后继续沿着记录组成的单向链表向后扫描。查找过程如下。

步骤 1. 确定第一条符合 c2=4 条件的目录项记录所在的页。

根据根页面（也就是页 44）可以快速定位到第一条符合 c2=4 条件的目录项记录所在的页为页 42（因为 2 < 4 < 9）。

步骤 2. 通过第一条符合 c2=4 条件的目录项记录所在的页面确定第一条符合 c2=4 条件的用户记录所在的页。

根据页 42 可以快速定位到第一条符合条件的用户记录所在的页为页 34 或者页 35 中（因为 2 < 4 ≤ 4）。

步骤 3. 在真正存储第一条符合 c2=4 条件的用户记录的页中定位到具体的记录。

到页 34 和页 35 中定位到具体的用户记录（如果在页 34 中使用页目录定位到第一条符合条件的用户记录，就不需要再到页 35 中使用页目录去定位第一条符合条件的用户记录了）。

步骤 4. 这个 B+ 树的叶子节点中的记录只存储了 c2 和 c1（也就是主键）两个列。在这个 B+ 树的叶子节点处定位到第一条符合条件的那条用户记录之后，我们需要根据该记录中的主键信息到聚簇索引中查找到完整的用户记录。这个通过携带主键信息到聚簇索引中重新定位完整的用户记录的过程也称为回表。然后再返回到这棵 B+ 树的叶子节点处，找到刚才定位到的符合条件的那条用户记录，并沿着记录组成的单向链表向后继续搜索其他也满足 c2=4 的记录，每找到一条的话就继续进行回表操作。重复这个过程，直到下一条记录不满足 c2=4 的这个条件为止。

为什么还需要一次回表操作呢？直接把完整的用户记录放到叶子节点不就好了么？你说得对，如果把完整的用户记录放到叶子节点是可以不用回表，但是太占地方了——相当于每建立一棵 B+ 树都需要把所有的用户记录复制一遍，这就太浪费存储空间了。

因为这种以非主键列的大小为排序规则而建立的 B+ 树需要执行回表操作才可以定位到完整的用户记录，所以这种 B+ 树也称为二级索引（Secondary Index）或辅助索引。由于我们是以 c2 列的大小作为 B+ 树的排序规则，所以我们也称这棵 B+ 树为为 c2 列建立的索引，把 c2 列称为索引列。二级索引记录和聚簇索引记录使用的是一样的记录行格式，只不过二级索引记录存储的列不像聚簇索引记录那么完整。

小贴士

我们上面把聚簇索引或者二级索引的叶子节点中的记录称为用户记录。为了区分，也把聚簇索引叶子节点中的记录称为完整的用户记录，把二级索引叶子节点中的记录称为不完整的用户记录。

另外，c1、c2 列存储的都是数字，为这两个列建立索引的过程比较好理解。如果我们为一个存储字符串的列建立索引，比如为 c3 列建立索引，情况会和上述过程有啥不一样么？没啥不一样，别忘了我们前面唠叨过的字符集和比较规则，字符串也是可以比较大小的。

3. 联合索引

我们也可以同时以多个列的大小作为排序规则，也就是同时为多个列建立索引。比如，我们想让 B+ 树按照 c2 和 c3 列的大小进行排序，这里面包含两层含义：

- 先把各个记录和页按照 c2 列进行排序；
- 在记录的 c2 列相同的情况下，再采用 c3 列进行排序。

为 c2 和 c3 列建立索引，示意图如图 6-15 所示。

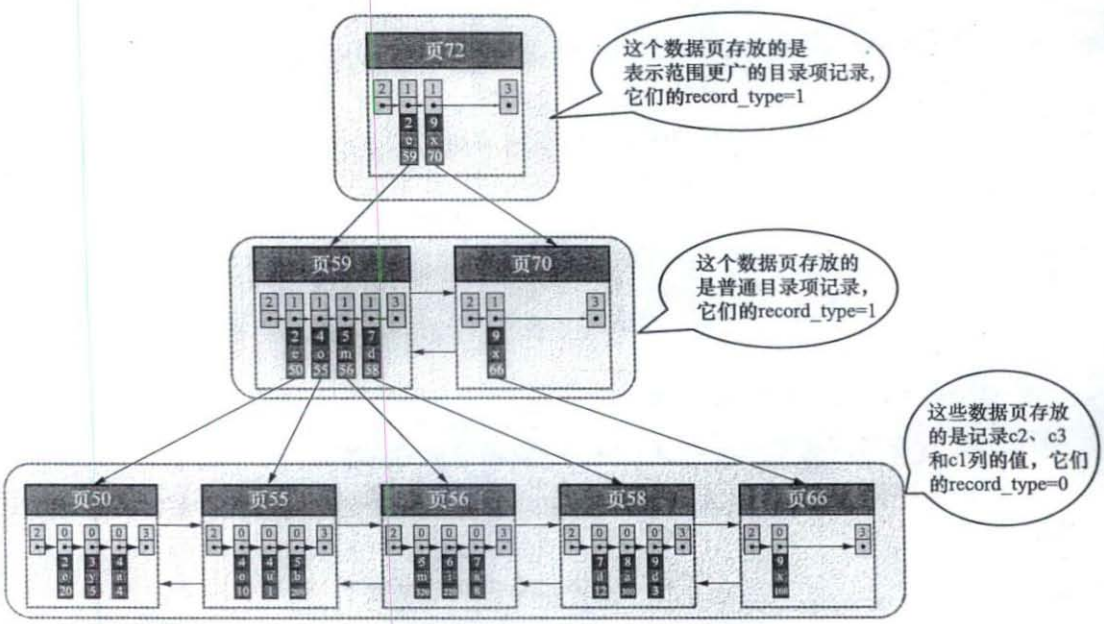

图 6-15 为 c2 和 c3 列建立的索引示意图

在图 6-15 中需要注意以下两点。

- 每条目录项记录都由 c2 列、c3 列、页号这 3 部分组成。各条记录先按照 c2 列的值进行排序，如果记录的 c2 列相同，则按照 c3 列的值进行排序。
- B+ 树叶子节点处的用户记录由 c2 列、c3 列和主键 c1 列组成。

千万要注意的是，以 c2 和 c3 列的大小为排序规则建立的 B+ 树称为联合索引，也称为复合索引或多列索引。它本质上也是一个二级索引，它的索引列包括 c2、c3。需要注意的是，“以 c2 和 c3 列的大小为排序规则建立联合索引”和“分别为 c2 和 c3 列建立索引”的表述是不同的，不同点如下。

- 建立联合索引只会建立如图 6-15 所示的一棵 B+ 树。
- 为 c2 和 c3 列分别建立索引时，则会分别以 c2 和 c3 列的大小为排序规则建立两棵 B+ 树。

6.2.3 InnoDB 中 B+ 树索引的注意事项

1. 根页面万年不动窝

前面在介绍 B+ 树索引的时候，为了方便理解，我们先把存储用户记录的叶子节点都画出来，然后再画出存储目录项记录的内节点。实际上 B+ 树的形成过程是下面这样的。

- 每当为某个表创建一个 B+ 树索引（聚簇索引不是人为创建的，它默认就存在）时，都会为这个索引创建一个根节点页面。最开始表中没有数据的时候，每个 B+ 树索引对应的根节点中既没有用户记录，也没有目录项记录。
- 随后向表中插入用户记录时，先把用户记录存储到这个根节点中。
- 在根节点中的可用空间用完时继续插入记录，此时会将根节点中的所有记录复制到一个新分配的页（比如页 a）中，然后对这个新页进行页分裂操作，得到另一个新页（比如页 b）。这时新插入的记录会根据键值（也就是聚簇索引中的主键值，或二级索引中对应的索引列的值）的大小分配到页 a 或页 b 中。根节点此时便升级为存储目录项记录的页，也就需要把页 a 和页 b 对应的目录项记录插入到根节点中。

在这个过程中，需要特别注意的是，一个 B+ 树索引的根节点自创建之日起便不会再移动（也就是页号不再改变）。这样只要我们对某个表建立一个索引，那么它的根节点的页号便会被记录到某个地方，后续凡是 InnoDB 存储引擎需要用到这个索引时，都会从那个固定的地方取出根节点的页号，从而访问这个索引。

小贴士

跟大家剧透一下，这个“存储某个索引的根节点在哪个页面中”的信息就是传说中的数据字典中的一项信息。关于数据字典的更多内容，第 9 章会详细唠叨，请别着急。

2. 内节点中目录项记录的唯一性

我们知道，在 B+ 树索引的内节点中，目录项记录的内容是索引列加页号的搭配，但是这个搭配对于二级索引来说有点儿不严谨。还是以 index_demo 表为例进行讲解，假设这个表中的数据如表 6-1 所示。

表 6-1 index_demo 表中的数据

c1	c2	c3
1	1	'u'
3	1	'd'
5	1	'y'
7	1	'a'

如果在二级索引中，目录项记录的内容只是索引列 + 页号的搭配，那么为 c2 列建立索引后的 B+ 树应该如图 6-16 所示。

如果我们想新插入一行记录，其中 c1、c2、c3 的值分别为 9、1、'c'，那么在修改为 c2 列建立的二级索引对应的 B+ 树时，便碰到了个大问题：由于页 3 中存储的目录项记录是由 c2 列 + 页号构成的，页 3 中的两条目录项记录对应的 c2 列的值都是 1，而新插入的这条记录中，c2 列的值也是 1，

那么这条新插入的记录到底应该放到页 4 中，还是应该放到页 5 中呢？答案是：对不起，发懵了。

为了让新插入的记录能找到自己在哪个页中，就需要保证 B+ 树同一层内节点的目录项记录除页号这个字段以外是唯一的。所以二级索引的内节点的目录项记录的内容实际上是由 3 部分构成的：

- 索引列的值；
- 主键值；
- 页号。

也就是我们把主键值也添加到二级索引内节点中的目录项记录中，这样就能保证 B+ 树每一层节点中各条目录项记录除页号这个字段外是唯一的，所以我们为 c2 列建立二级索引后的示意图实际上应该如图 6-17 所示。

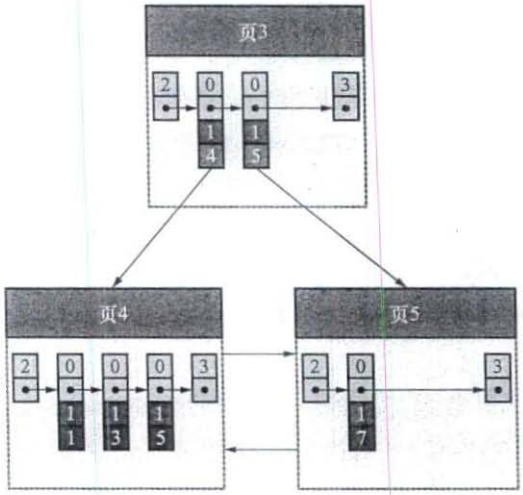

图 6-16　为 c2 列建立索引后的 B+ 树

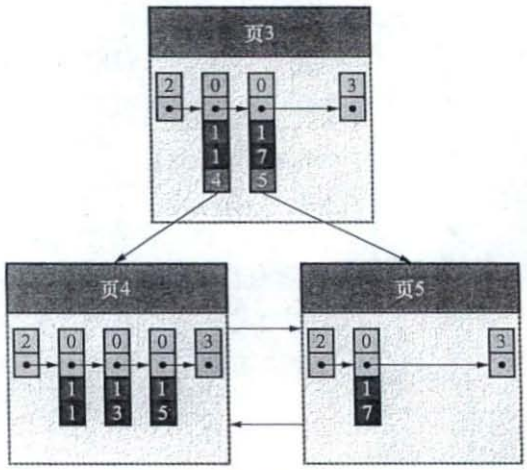

图 6-17　二级索引内节点的目录项记录实际包含主键值

这样我们再插入记录 (9, 1, 'c') 时，由于页 3 中存储的目录项记录是由 c2 列 + 主键 + 页号构成的，因此可以先把新记录的 c2 列的值和页 3 中各目录项记录的 c2 列的值进行比较；如果 c2 列的值相同，可以接着比较主键值。因为 B+ 树同一层中不同目录项记录的 c2 列 + 主键的值肯定是不一样的，所以最后肯定能定位到唯一的一条目录项记录。

在本例中，最后确定新记录应该插入到页 5 中。

小贴士

对于二级索引记录来说，是先按照二级索引列的值进行排序，在二级索引列值相同的情况下，再按照主键值进行排序。所以，为 c2 列建立索引其实相当于为 (c2, c1) 列建立了一个联合索引。另外，对于唯一二级索引（当我们为某个列或列组合声明 UNIQUE 属性时，便会为这个列或列组合建立唯一二级索引）来说，也可能会出现多条记录键值相同的情况（一是声明为 UNIQUE 属性的列可能存储多个 NULL 值，二是我们后面要讲的 MVCC 服务），唯一二级索引的内节点的目录项记录也会包含记录的主键值。

3. 一个页面至少容纳 2 条记录

前面说过，一棵 B+ 树只需要很少的层级就可以轻松存储数亿条记录，查询速度杠杠的！这是因为 B+ 树本质上就是一个大的多层级目录，每经过一个目录时都会过滤掉许多无效的子目录，直到最后访问到存储真正数据的目录。

如果一个大的目录中只存放一个子目录是啥效果呢？那就是目录层级会非常多，而且最后那个存放真正数据的目录中只能存放一条记录。费了半天劲只能存放一条真正的用户记录？逗我玩？所以 InnoDB 的一个数据页至少可以存放 2 条记录，这也是我们之前唠叨记录行格式的时候说过的一个结论（我们当时以这个结论为基础，推导了表中只有一个列且该列在不发生溢出的情况下，最多能存储多少字节。大家如果忘了的话请回过头去看看吧）。

小贴士

其实，让 B+ 树的叶子节点只存储一条记录，让内节点存储多条记录，也还是可以发挥 B+ 树作用的。但是，设计 InnoDB 的大叔还是为了避免 B+ 树的层级增长得过高，而要求所有数据页都至少可以容纳 2 条记录。

6.2.4　MyISAM 中的索引方案简介

至此，我们介绍的都是 InnoDB 存储引擎中的索引方案。为了内容的完整性，以及各位同学可能会在面试时遇到这类问题，我们还是有必要简单介绍一下 MyISAM 存储引擎中的索引方案。

我们知道，在 InnoDB 中索引即数据，也就是聚簇索引的那棵 B+ 树的叶子节点中已经包含了所有完整的用户记录。MyISAM 的索引方案虽然也使用树形结构，但是却将索引和数据分开存储。

- 将表中的记录按照记录的插入顺序单独存储在一个文件中（称之为数据文件）。这个文件并不划分为若干个数据页，有多少记录就往这个文件中塞多少记录。这样一来，我们可以通过行号快速访问到一条记录。

MyISAM 记录也需要记录头信息来存储一些额外数据。我们以前文唠叨过的 index_demo 表为例，看一下这个表在使用 MyISAM 作为存储引擎时，它的记录如何在存储空间中表示，如图 6-18 所示。

0	记录头	1	4	u
1	记录头	3	9	d
2	记录头	5	3	y
3	记录头	4	4	a
4	记录头	100	9	x
5	记录头	8	7	a
6	记录头	209	5	b
7	记录头	300	8	a
8	记录头	20	2	e
9	记录头	10	4	o
10	记录头	12	7	d
11	记录头	220	6	i
12	记录头	320	5	m

图 6-18　index_demo 表使用 MyISAM 作为存储引擎在存储空间中的表示

由于在插入数据时并没有刻意按照主键大小排序，所以我们不能在这些数据上使用二分法进行查找。

- 使用 MyISAM 存储引擎的表会把索引信息单独存储到另外一个文件中（称为索引文件）。MyISAM 会为表的主键单独创建一个索引，只不过在索引的叶子节点中存储的不是完整的用户记录，而是主键值与行号的组合。也就是先通过索引找到对应的行号，再通过行号去找对应的记录！

这一点与 InnoDB 是完全不相同的。在 InnoDB 存储引擎中，我们只需要根据主键值对聚簇索引进行一次查找就能找到对应的记录，而在 MyISAM 中却需要进行一次回表操作，这也意味着 MyISAM 中建立的索引相当于全部都是二级索引！

- 如果有必要，我们也可以为其他列分别建立索引或者建立联合索引，其原理与 InnoDB 中的索引差不多，只不过在叶子节点处存储的是相应的列 + 行号。这些索引也全部都是二级索引。

小贴士

MyISAM 的行格式有定长记录格式（Static）、变长记录格式（Dynamic）、压缩记录格式（Compressed）等。上文用到的 index_demo 表采用定长记录格式，也就是一条记录占用的存储空间是固定的，这样就可以使用行号轻松算出某条记录在数据文件中的地址偏移量了。但是变长记录格式就不行了，MyISAM 会直接在索引叶子节点处存储该条记录在数据文件中的地址偏移量。由此可以看出，MyISAM 的回表操作是十分快速的，因为它是拿着地址偏移量直接到文件中取数据，而 InnoDB 是通过获取主键之后再去聚簇索引中找记录，虽然说也不慢，但还是比不上直接用地址去访问。

这里只是非常简要地介绍了 MyISAM 的索引，要是将具体细节全部写出来就又可以独立成章了。这里只是希望大家理解 InnoDB 中的“索引即数据，数据即索引”，而在 MyISAM 中却是“索引是索引，数据是数据”。

6.2.5 MySQL 中创建和删除索引的语句

前文光顾着唠叨索引的原理了，我们如何使用 MySQL 语句建立这种索引呢？ InnoDB 和 MyISAM 会自动为主键或者带有 UNION 属性的列建立索引。如果想为其他的列建立索引，就需要我们显式地指明了。为啥不自动为每个列都建立索引呢？别忘了，每建立一个索引都会建立一棵 B+ 树，而且每增、删、改一条记录都要维护各个记录、数据页的排序关系，这是很费性能和存储空间的。

我们可以在创建表的时候，指定需要建立索引的单个列或者建立联合索引的多个列：

```
CREATE TALBE 表名 (
    各个列的信息 ··· ,
    (KEY|INDEX) 索引名 (需要被索引的单个列或多个列)
)
```

其中，KEY 和 INDEX 是同义词，任意选用一个就可以。

我们也可以在修改表结构的时候添加索引：

```
ALTER TABLE 表名 ADD (INDEX|KEY) 索引名 (需要被索引的单个列或多个列);
```

还可以在修改表结构的时候删除索引：

```
ALTER TABLE 表名 DROP (INDEX|KEY) 索引名;
```

比如，我们想在创建 index_demo 表时就为 c2 和 c3 列添加一个联合索引，可以这么写建表语句：

```
CREATE TABLE index_demo(
    c1 INT,
    c2 INT,
    c3 CHAR(1),
    PRIMARY KEY(c1),
    INDEX idx_c2_c3 (c2, c3)
);
```

在这个建表语句中，创建的索引的名称是 idx_c2_c3。索引的名字尽管可以随意起，不过还是建议在命名时能以 idx_ 为前缀，后面跟着需要建立索引的列名，且多个列名之间用下划线分隔开。

如果我们想删除这个索引，可以这么写：

```
ALTER TABLE index_demo DROP INDEX idx_c2_c3;
```

6.3 总结

InnoDB 存储引擎的索引是一棵 B+ 树，完整的用户记录都存储在 B+ 树第 0 层的叶子节点；其他层次的节点都属于内节点，内节点中存储的是目录项记录。

InnoDB 的索引分为两种。

- 聚簇索引：以主键值的大小作为页和记录的排序规则，在叶子节点处存储的记录包含了表中所有的列。
- 二级索引：以索引列的大小作为页和记录的排序规则，在叶子节点处存储的记录内容是索引列 + 主键。

InnoDB 存储引擎的 B+ 树根节点自创建之日起就不再移动。

在二级索引的 B+ 树内节点中，目录项记录由索引列的值、主键值和页号组成。

一个数据页至少可以容纳 2 条记录。

MyISAM 存储引擎的数据和索引分开存储，这种存储引擎的索引全部都是二级索引，在叶子节点处存储的是列 + 行号（对于定长记录格式的记录来说）。